Exercise 05: *PyUVM* Test

Objective: *Introduction to PyUVM tests.*

Task: *Development of PyUVM tests using the Saturation Filter.*

In this exercise you need to develop simple *PyUVM* tests for a saturation filter (SAT Filter). First make yourself familiar with the DUT. Do not look at the RTL, but rather read the provided documentation in `<ROOT>/lecture03/exercises/E05_sat_pyuvm_test/exercise_general_info.pdf`.

Part 1: Simple Test

`<ROOT>/lecture03/exercises/E05_sat_pyuvm_test/tb` contains a `Makefile` which already correctly imports the SAT filter RTL from `<ROOT>/lecture03/exercises/E05_sat_pyuvm_test/rtl`. You only need to update `COCOTB_TEST_MODULES` with your test module name.

Reimplement the simple test from Part 1 of Exercise 04 as a *PyUVM* test in `sat_default_test.py`. Keep the same clock, reset, stimulus, and output checks, but organize the test as follows:

- Name the test class `sat_default_test`.
- Extend `uvm_test` and decorate the class with `@pyuvm.test()`.
- Get the DUT handle in the `build_phase`.
- Include an empty `connect_phase`.
- Copy the test behavior from Exercise 04's `sat_default_test.py` into the `run_phase`. Raise an objection before the test starts and drop it when the test ends.

Follow the same sSDT protocol requirements as in Exercise 04. See `<ROOT>/lecture03/exercises/E05_sa` for more information about the protocol. Add `sat_default_test` to the `Makefile`'s `COCOTB_TEST_MODULES` variable.

Run the test using the `Makefile` and inspect the waveforms.

Part 2: Random Test

Reimplement the random test from Part 2 of Exercise 04 as a separate *PyUVM* test in `sat_random_test.py`. Use `sat_default_test.py` as a starting point and name the test class `sat_random_test`. Copy the test behavior from Exercise 04's `sat_random_test.py` into the `run_phase`, and keep the *PyUVM* phase structure and objections from Part 1.

Add `sat_random_test` to the `Makefile`'s `COCOTB_TEST_MODULES` variable. Run the test and inspect the test results and waveforms.

Part 3: Interface Wrapper Test

Create a third *PyUVM* test in the file `sat_interface_test.py`. Use `sat_default_test.py` as a starting point (copy it), but access the DUT signals through instances of the following interface wrapper:

The SAT filter has one input and one output sSDT interface. See `<ROOT>/lecture03/exercises/E05_sat_`

```python
class ssdt_interface_wrapper():
    def __init__(self, clk=None, rst=None, name="ssdt_interface"):
        self.name = name
        self.clk = clk
        self.rst = rst
        self.valid = None
        self.data = None

    def connect(self, clk_signal, reset_signal,
                valid_signal, data_signal):
        self.clk = clk_signal
        self.rst = reset_signal
        self.valid = valid_signal
        self.data = data_signal
```

Listing 0.1: Interface wrapper for an sSDT interface.

for more information about the sSDT interfaces and their signals. Create two `ssdt_interface_wrapper` instances after getting the DUT handle in the `build_phase`: one for the input interface and one for the output interface.

In the `connect_phase`, connect:

- The input wrapper to `clk`, `rst`, `in_valid`, and `in_data`.
- The output wrapper to `clk`, `rst`, `out_valid`, and `out_data`.

Name the test class `sat_interface_test`. Its `run_phase` must perform the same reset, stimulus, and checks as `sat_default_test`, but all signal accesses must go through the interface wrappers instead of the DUT handle.

Add `sat_interface_test` to the `Makefile`'s `COCOTB_TEST_MODULES` variable. Run all three tests and inspect the test results and waveforms.